# An Open, Native-Linux Reproduction of the Canon MegaTank 5B00 Waste-Ink Reset: Reverse Engineering the usbprint Vendor Transport and the Session-Keyword Write Cipher

J. Sullivan
2026

***Abstract*—Canon MegaTank (G-series) inkjet printers permanently disable themselves with support code 5B00 once an internal waste-ink absorber counter saturates, reporting only "service is required" with no user-accessible reset. On the G5000/G6000/G7000 generation the classic service-mode clear is disabled in firmware, so the only working remedy is a commercial, cloud-gated Windows tool that validates a single-use, per-printer key against a vendor server. We present a fully open, native-Linux, key-free, and cloud-free reproduction of that reset, derived through a trace↔decompile↔correlate methodology: host `usbmon` wire capture, Frida instrumentation of the Windows tool inside a virtual machine, and Ghidra decompilation of both the vendor tool and Windows' `usbprint.sys`. We recover the transport (a USB *vendor* control transfer, `0x41` OUT / `0xC1` IN, with `bRequest` carrying the command byte), the session protocol (a plain `set_session`, a live per-session device keyword read via `get_keyword`, and an enciphered `set_command` write), and the write cipher itself—a functor envelope whose buffer roles are swapped relative to the naive reading, validated byte-exact (23/23) against a genuine captured frame. To obtain that ground-truth frame we bypass the tool's three cloud-licensing gates with a one-byte-per-gate `JZ`→`JMP` patch, and we prove by decompilation that no cloud byte ever reaches the reset payload, keyword, or completion: the cloud is licensing only. The resulting native tool clears 5B00 on real hardware (Canon device `04a9:1865`/service `04a9:12fe`) using only libusb and one live keyword read, behind a stack of safety gates (test-unit isolation, EEPROM dump, write budget, lockfile). We additionally document an operationally critical nuance: the cleared counter commits only on a clean power-button shutdown (printhead-park EEPROM flush), not on an abrupt unplug.**

***Index Terms*—Canon MegaTank, waste-ink reset, 5B00, USB reverse engineering, usbprint vendor control transfer, session-keyword cipher, DRM bypass, Frida, Ghidra, right to repair**

## I. Introduction

Canon MegaTank printers route stray ink—from cleaning cycles, borderless printing, and power-on priming—into an internal absorber pad. Firmware maintains a software counter that estimates absorber saturation, and when that counter crosses a threshold the printer halts with support code 5B00 and the message "service is required." Canon offers no user reset; official guidance is warranty replacement or disposal [1]. On earlier PIXMA models a panel key combination cleared the counter, but on the G5000/G6000/G7000 generation that built-in reset is removed in firmware: the printer still *enters* service mode, but accepts and silently ignores the legacy clear command.

In practice the only path back to a working printer on this generation is a commercial Windows utility (WICReset / Printer Potty) that requires a single-use, per-printer key validated online against a vendor server [2]. This is the canonical software-counter-as-DRM situation: a mechanically serviceable device is bricked by a counter, and the unlock is gated behind a proprietary, network-dependent, paid tool. Replacing or externally plumbing the physical absorber is straightforward; clearing the firmware counter is the hard part.

**Our contribution** is an open, native-Linux, key-free, and cloud-free reproduction of the MegaTank 5B00 reset, together with the reverse-engineering evidence that makes it reproducible. Specifically:

1) We recover the service-mode *transport* by decompiling Windows' `usbprint.sys`: the maintenance commands are USB *vendor* control transfers, not bulk transfers (Section V).
2) We document the session *protocol*—plain `set_session`, a live per-session device keyword from `get_keyword`, and the enciphered `set_command` writes—and characterize the empty `get_command` completion (Section VI).
3) We crack the write *cipher*: a functor envelope whose buffer roles (subject vs. seed) are swapped relative to the obvious reading, validated 23/23 against a genuine captured frame (Section VII).
4) We obtain that genuine frame by bypassing three cloud-licensing gates and *prove by decompilation* that the reset is cloud-independent (Section VIII).
5) We reproduce the reset natively on real hardware and document the power-button commit nuance (Section IX), behind a safety-gate stack (Section X) with hardware results (Section XI).

All findings, scripts, and the reference codec are open source under a clean-room, interoperability, and right-to-repair framing (Sections XIII and XV) [3].

## II. Background

### A. Canon MegaTank service mode

A MegaTank printer enumerates differently depending on mode. In normal operation the G6020 is USB device `04a9:1865` with several interfaces. Entering service mode (a panel power/resume key sequence) re-enumerates the printer as `04a9:12fe`: a single printer-class interface (interface 0, alternate 0) whose IEEE-1284 device ID advertises `MFG:Canon;CMD:BJL,...;PSE:KMDA10021`. Maintenance commands are issued over this service-mode interface. Crucially, on this generation the firmware accepts the legacy clear and does nothing with it—the gate is not a different opcode but a refusal to honor an unauthenticated write (Section III).

### B. The pixma firmware lineage

Open Canon reverse engineering descends from the Context IS "doomed encryption" work on PIXMA firmware updates [4] and the `leecher1337/pixma` toolkit [5], which recovers a firmware XOR key from a known SREC plaintext and unpacks a Thumb-compressed payload. That lineage is a *firmware-unpack* toolkit: it contains no service-mode, USB-framing, or waste-counter logic, and ships without a license. The SANE `pixma` scanner backend [6] shows Canon's command-framing style but operates on a bulk channel unrelated to the maintenance transport. The open Epson lineage is more directly transferable in spirit: tools such as `reink` [7], `epson_print_conf` [8], and `epson-printer-snmp` [9] reset Epson maintenance counters, the last by sniffing WICReset—the same capture-and-correlate method we apply to Canon. No open Canon G-series equivalent existed prior to this work.

### C. usbprint vendor control transfers

Windows binds the service-mode interface to the `usbprint.sys` printer minidriver, which exposes private IOCTLs. Two are load-bearing here: `IOCTL_USBPRINT_VENDOR_SET_COMMAND` (`0x220038`) and `VENDOR_GET_COMMAND` (`0x22003c`). As we show in Section V, each maps to a single USB *vendor*-class, interface-recipient control transfer on endpoint 0—not to a bulk transfer on the printer's data endpoints. This distinction is the reason every prior native attempt stalled.

## III. Problem Analysis

### A. Symptom characterization

The printer reports 5B00 and refuses to print. Over IPP the printer-state is `stopped` with a `marker-waste-ink-receptacle-full` alert:

```
$ ipptool -tv ipp://printer/ get-attrs.test
  printer-state = stopped (5)
  printer-state-reasons = marker-waste-ink-
      receptacle-full
```

Listing 1. 5B00 surfaces as an IPP printer-state alert.

The legacy service-mode clear is accepted on the wire but does not persist. Listing 2 shows the falsified attempt: a well-formed vendor control-OUT carrying the old payload returns an ACK, yet 5B00 survives a power cycle.

```
# vendor control-OUT, legacy payload -> device ACK
      5
ctrl(0x40, 0x85, 0x0000, 0x0000, [00 03 01 03 07])
# power-cycle -> printer-state still "stopped", 5
     B00 returns
```

Listing 2. Legacy clear is ACKed but does not clear 5B00 (falsified, multiple sessions).

### B. The firmware authorization barrier

Table I summarizes the falsification campaign. The pattern is consistent: every unauthenticated frame—over bulk or control, with the checkbox flag set or not, with the index in the payload or in `wValue`—is accepted (ACK) but ignored (5B00 persists). The working commercial tool differs not in the opcode but in opening an authenticated *session* keyed to a live device value before writing.

TABLE I
Falsified Reset Attempts (G6020, service mode `12fe`)

| Attempt | Channel | Outcome |
|---|---|---|
| Bare `85 00 00 00 03 01 03 07` | bulk OUT `0x01` | ACK; 5B00 persists |
| Legacy payload, flag `0x81` | vendor ctrl-OUT | ACK 5; 5B00 persists |
| Index in `wValue` | vendor ctrl-OUT | ACK; 5B00 persists |
| Plain `set_session 0x81` | vendor ctrl-OUT | STALL |
| Bulk-IN read `0x82` | bulk IN | ZLP (0 bytes) |

Authenticated session (Section VI) is the only path that clears.

## IV. Method: The Trace–Decompile–Correlate Trifecta

Our methodology triangulates three independent evidence sources so that each closes the others' blind spots (Figure 1). *Trace*: host `usbmon` captures the literal wire URBs at the bus, which a TLS-encrypted in-process view cannot fake. *Decompile*: Ghidra recovers the intended semantics—the IOCTL→URB mapping in `usbprint.sys` and the frame builder/cipher in the vendor tool—which opaque ciphertext on the wire cannot reveal. *Correlate*: Frida hooks `DeviceIoControl` inside the tool to expose plaintext in/out buffers with timestamps, pairing each decompiled routine to the exact URB it produced. The phases below are executed within one VM session against one genuine reset so the three layers describe the *same* event.

The capture environment is a Win11 guest with the printer passed through via QEMU USB passthrough; per prior lab notes the guest requires an e1000e NIC and NTLM WinRM transport for reliable automation. We treat the VM as equivalent to bare metal for capture purposes: the device sits on the real bus and the wire bytes are identical.

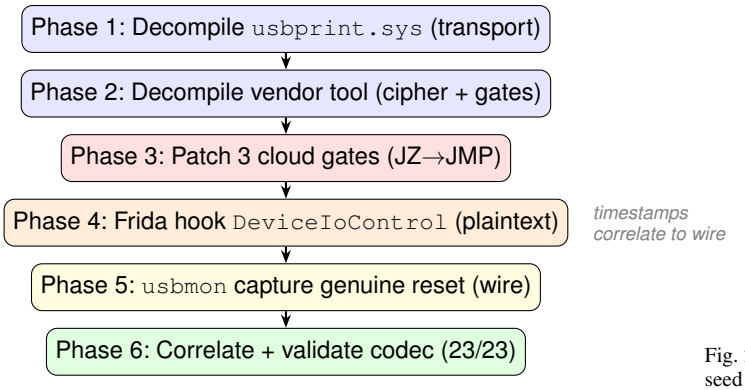



Fig. 1. The trace↔decompile↔correlate trifecta. Phases 1–2 recover intended semantics; phases 3–5 capture one genuine reset at the IOCTL and wire layers simultaneously; phase 6 proves the native codec reproduces the captured frame byte-exact.

## V. Transport Reverse Engineering

We decompiled `usbprint.sys` (Win11 24H2, `10.0.26100.8328`) and traced the device-control dispatch to the `VENDOR_SET`/`VENDOR_GET` handlers. Both allocate a `_URB_CONTROL_VENDOR_OR_CLASS_REQUEST` (function `0x0018`, `URB_FUNCTION_VENDOR_INTERFACE`) and submit it to the bus driver—there is no bulk path. The handler reads the IOCTL input buffer and assigns the URB fields as follows: `bRequest = inBuf[0]; wValue = (inBuf[1] « 8) | inBuf[2]; wIndex = (bInterfaceNumber « 8) | bAlternateSetting`; and—decisively—the *entire* input buffer becomes the data stage (`TransferBuffer = SystemBuffer, TransferBufferLength = InputBufferLength`). The driver does *not* strip the three header bytes: `inBuf[0]` seeds `bRequest` *and* remains the first data byte.

Figure 2 shows the resulting on-wire control frame. `VENDOR_SET` yields `bmRequestType = 0x41` (vendor, interface, host→device); `VENDOR_GET` yields `0xC1` (vendor, interface, IN). This was cross-checked against two mappings the live device already confirmed: `GET_1284_ID` (class/interface/IN ⇒ `0xA1`) and `VENDOR_GET` of the model/version string (`0xC1/0x8a`), both byte-exact.

The endianness of `wValue` is explicit in the decompile (`inBuf[1]` is the high byte); all target frames carry argument bytes `00 00`, so `wValue = 0x0000` regardless, but a future non-zero argument must follow the decompiled order. The native transport is a thin libusb wrapper: `send_command` issues a `0x41` control-OUT with the whole frame as data; `send_and_receive` issues a `0xC1` control-IN for a bare 3-byte read header and the read length as `wLength` (Section VI).

## VI. The Wire Protocol and the Per-Session Keyword

The reset is an ordered session of four logical commands over the vendor transport (Figure 3). The command

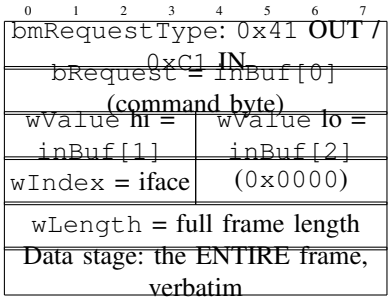



Fig. 2. usbprint vendor control frame. The command byte and argument bytes seed the setup packet *and* remain at the head of the data stage—the driver never strips `inBuf[0..2]`. Splitting the header off the data stage was the cause of every prior control-pipe STALL.

byte rides in `bRequest`: `0x81 set_session`, `0x82 get_keyword`, `0x85 set_command`, `0x86 get_command`.

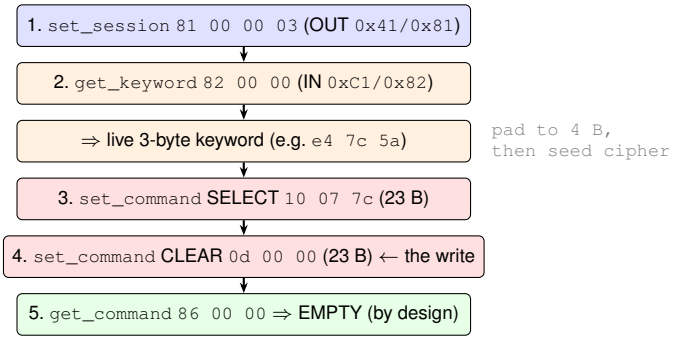



Fig. 3. The service-mode reset state machine. The live keyword from step 2 is the only runtime input; it keys the two enciphered `set_command` writes to the session. The `get_command` read-back is empty on this device and must not be used as a success gate.

**The live keyword.** `get_keyword` returns a 3-byte per-session value (e.g. `e4 7c 5a`). The cipher seed is 4 bytes, so the reply is right-padded to `e4 7c 5a 00` before seeding the encoder. This read *must* precede the writes: it binds them to the live session. We observed (and reproduced) that a `get_version` prime `8a 00 00` returns a 20-byte enciphered reply that decrypts locally to the model string "Canon G6000 series" with zero cloud interaction—direct corroboration that the device, not the cloud, sources session state.

**The empty completion.** `get_command` (`86 00 00`) returns an empty reply on this device, and there is no finalize command. Code that gates success on a non-empty `0x86` read will wrongly fail a successful reset; the clear is committed by the two writes plus a clean power-off (Section IX), not by `0x86`.

## VII. The Write Cipher

### A. Frame shape

A genuine `set_command` is a single 23-byte frame: the 3-byte header `85 00 00` followed by a 20-byte ciphertext

(Figure 4). It is *not* a select/clear pair, and not the naive "header ‖ 4-byte enciphered keyword" shape that an early model emitted. The genuine captured payload for the SELECT operand is `db bb 00 67 59 a1 b0 1f 84 2f d5 83 04 4a 3a c3 51 d2 b1 ef`: 20 distinct, high-entropy bytes, exactly one zero, with no `00 12 01` envelope marker visible on the wire.

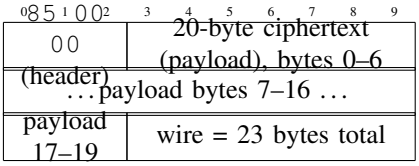



Fig. 4. The 23-byte `set_command` frame: header `85 00 00` ‖ 20-byte enciphered payload. The whole frame is the control-OUT data stage; the header must never be split from the payload.

### B. The functor buffer-role swap

The cipher is built from two functors recovered from the vendor tool's decompile. A functor-3 step deterministically expands the application command into a 20-byte *envelope*; a functor-2 step then transforms that envelope under a 4-byte *bound keyword*. The non-obvious correction—the crux of the crack—is that the buffer roles are swapped relative to the naive reading: the 20-byte envelope is the SUBJECT of functor-2 and the bound keyword is the SEED, not the reverse. Listing 3 states the validated construction.

```
app    = b"\x85\x00\x00" + operand # e.g. 10 07 7c (SELECT)
envelope = envelope3(method=3, app) # 20-byte envelope
bound  = bind_keyword(method, kw_pad4) # 4-byte session bind
# SUBJECT = envelope (20B), SEED = bound keyword (4B):
payload = functor2(method, envelope,
                   seed=bound, send=True) # 20 bytes
wire   = b"\x85\x00\x00" + payload # 23 bytes
```

Listing 3. The validated CANON-SR5 write construction (functor-3 envelope, functor-2 keyword bind, role-swapped).

### C. 23/23 validation

For the genuine captured live keyword `e4 7c 5a` (padded `e4 7c 5a 00`, bound `00 35 a9 09`), both independent code paths—a parser-driven encoder loaded from the recovered template and a from-scratch reimplementation of the decompiled transform—reproduce the captured wire frames byte-exact (Listing 4). This is the central positive result: 23/23 bytes per frame, on the genuine session keyword, from two non-shared implementations.

```
SELECT set_command 10 07 7c
 850000 dbbb006759a1b01f842fd583044a3ac351d2b1ef
CLEAR set_command 0d 00 00 <- the 5B00 write
 850000 4dbb006759a1b01f842fd58319a83a627bafb1ef
```

Listing 4. Byte-exact (23/23) reproduction of the genuine captured frames for keyword `e4 7c 5a`.

We also report a deliberate *negative* result. The keyword reply is 3 bytes but the seed is 4 bytes, and the correct 4th-byte padding rule cannot be inferred from a single `(keyword, payload)` pair: the payload is a full-entropy, operand- and keyword-dependent ciphertext, and one sample cannot separate the keyword keystream from the operand dependence. The cipher reproduces the captured frame exactly and is keyword-parametric (it also reproduces a differently-keyed prior capture), but generalizing the padding rule to arbitrary keywords requires a controlled multi-sample campaign, analogous to the ∼40-sample campaign that cracked the read-side codec. We state this boundary explicitly rather than overclaim—and we close it in Section VII-D, where a ∼550-keyword read-only campaign confirms the keyword-parametric encoder byte-exact.

### D. Multi-sample confirmation via the `0x8c` readback

The boundary above—that one `(keyword, payload)` pair cannot generalize the keyword bind and 4th-byte padding—is resolved by an unexpected property of the service-mode readback. We captured 540 read-only sessions of constant device state (open session, read the live keyword, read registers `0x84` and `0x8c`; no writes), each under a distinct session keyword. Register `0x84` decodes to a constant 20-byte device descriptor under a linear keyword-XOR stream. Register `0x8c` is nonlinear in all three keyword bytes and at first appears to be the live counter—it varies every session. It is not: `0x8c` is the printer echoing its *own functor-2 write keystream over a zero plaintext*, i.e. `functor2(method3, 0^20, seed=bind_keyword(method3, kw‖`. The encoder that emits the clear reproduces all 540 `0x8c` replies with zero conflicts and predicts 16 further *out-of-sample* sessions byte-exact *before* reading them.

Two consequences follow. First, the keyword bind and 4-byte padding rule are *confirmed for arbitrary keywords*: the multi-sample campaign the negative result called for has been run, and the keyword-parametric encoder is byte-exact across ∼550 distinct keywords. Register `0x8c` is, in effect, a free on-hardware oracle for the write keystream—the strongest validation of the clear path short of issuing the clear hundreds of times. Second, it is a cautionary methodological result: the register that *looked* most like the counter, precisely because it varied independently of state, was a keyed keystream echo carrying no counter data. The actual counter is reached through the template's `query.normal` select path (`[10 07 7C][15]`), not a bare register read.

## VIII. DRM Bypass for Ground Truth, and the Cloud-Independence Proof

### A. Three cloud gates

To capture a genuine frame we had to run the commercial tool to the point of emitting the reset, which requires passing its online licensing. The reset orchestrator (`Core::ActionCanonDeviceClearCounters`) reaches the net-free emit routine (`clearCounters`) only

through three cloud round-trips, each guarded by a conditional jump (Figure 5): RESET_GUID, then QUERY_KEYS transport-success, then a QUERY_KEYS validity bit. In an offline VM all three stall on the licensing socket, and the tool aborts before any USB write.

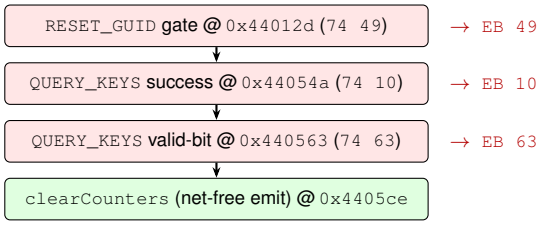



Fig. 5. The three cloud-licensing gates on the path to the net-free emit. Each is forced taken by rewriting the conditional jump JZ (0x74) to an unconditional JMP (0xEB), preserving the displacement byte so the target is unchanged. Patching only the two QUERY_KEYS gates is insufficient: RESET_GUID runs first and stalls the same way.

The patch is one byte per gate (Listing 5), applied at spawn via Frida so it is in place before the Reset button is clicked. Each rewrite keeps the relative displacement, so the unconditional jump lands on the same target the original would on success. A guard aborts loudly if the opcode is not 0x74, since the offsets are exact only for this binary hash; the image is ASLR-enabled, so a runtime slide is computed.

```
function jzToJmp(addr) {
  const p = va(addr);          // static VA + ASLR slide
  if (p.readU8() !== 0x74) { // guard: hash-specific offsets
    console.log("[!] expected 0x74 -- ABORT");
      return;
  }
  Memory.patchCode(p, 1, code => // W^X-safe, flushes i-cache
    code.writeU8(0xEB));
}
jzToJmp("0x44012d"); // RESET_GUID
jzToJmp("0x44054a"); // QUERY_KEYS success
jzToJmp("0x440563"); // QUERY_KEYS valid-bit
```

Listing 5. Forcing a licensing gate taken: JZ→JMP (0x74→0xEB), displacement preserved.

### B. No cloud byte reaches the reset

The decompile proves the reset is cloud-independent. QUERY_KEYS collapses its entire network reply to a single boolean (a one-byte memmove into a bool); that buffer is never read again. clearCounters is invoked with *no* argument—no token, seed, or buffer from any cloud reply is threaded in. The reset bytes are built entirely locally: the per-model template is a bundled PE resource (an APP.BIN blob that decrypts—3DES-EDE3-CBC under an all-zero key and IV, pure obfuscation—to a devices.xml model database), and the only runtime input is the live keyword read over USB. A call-graph traversal confirms the clearCounters subtree is net-free. The cloud's three roles are licensing only: a key-validation boolean (QUERY_KEYS), an optional device-list refresh, and a post-reset accounting upload (RESET_DATA, after the USB write). None sources a device-bound byte. The residual caveat is that the licensing bodies are TLS on the wire, so the in-process decompile is the proof; the simultaneous usbmon capture is the empirical backstop, and it shows the emitted frames are byte-identical to the locally-derived template.

## IX. Native Reproduction and the Commit Nuance

The native tool needs no key, no cloud, no Windows, and no VM: it opens the service-mode device, reads the live keyword, enciphers the two set_command writes with the recovered codec, and issues them over the libusb vendor transport. Listing 6 shows the gated execute path.

```
# dry-run (default): prints every wire frame,
    touches no USB
$ canon-megatank reset-native
# execute (passes the full safety-gate ladder,
    Section IX):
$ canon-megatank reset-native --execute --accept-
    derived
  reset_native.ok keyword=e4 7c 5a 00 steps
      =[81,82,85,85,86]
  commit_step: release USB, then CLEAN POWER-BUTTON
       shutdown
```

Listing 6. Native reset (gated execute path).

**The power-button commit (critical).** The two writes do not persist the cleared counter by themselves. The reset commits only on a *clean power-button shutdown*, in order: (1) release the USB handle; (2) press the printer's power button for a normal shutdown. The clean shutdown lets the printhead park and the firmware flush the cleared EEPROM page. An abrupt unplug skips the park-and-flush, the page is not written back, and 5B00 returns on the next boot. This was the single most important hardware lesson of the validated run, and it matches the commercial tool's own vendor instruction to power off via the button immediately after the success window.

## X. Safety Gates

Because this is a destructive EEPROM write on an owned device, every -execute runs an ordered gate ladder; if any gate refuses, no USB is touched (Table II). The default mode is a dry-run that prints all wire frames and the commit step without opening the device.

TABLE II
Execute-Path Safety Gate Ladder (in order)

| # | Gate | Refusal effect |
|---|---|---|
| 1 | Test-unit UUID isolation | wrong unit ⇒ refuse |
| 2 | Validation status | not verified ⇒ stop* |
| 3 | EEPROM dump present | no baseline ⇒ refuse |
| 4 | Write-budget charge | over cap ⇒ refuse |
| 5 | Lockfile (per serial) | concurrent run ⇒ refuse |
| 6 | Live-keyword guard | short read ⇒ stop pre-write |

*A logged one-run -accept-derived override does not mutate the SSOT.

Gate 1 fingerprints the unit against a locked test-unit UUID. Gate 2 requires the protocol status to be promoted to "verified" before unattended execution; the shipped status is deliberately

"derived-unvalidated," so execution requires an explicit, loudly logged one-run override. Gate 3 requires a pre-flight EEPROM dump as a rollback baseline. Gate 6 hard-stops before any write if `get_keyword` returns fewer than the minimum bytes, so a degenerate session never proceeds to a write.

## XI. Results

### A. Initial analysis

Table III records the device identity recovered over the vendor transport before any write.

TABLE III
SERVICE-MODE DEVICE IDENTITY (G6020)

| Property | Value |
|---|---|
| Normal-mode USB id | `04a9:1865` |
| Service-mode USB id | `04a9:12fe` (iface 0, alt 0) |
| IEEE-1284 ID | `MFG:Canon;...;PSE:KMDA10021` |
| Model (decrypted) | "Canon G6000 series", `CANON-SR5` |
| Maintenance method | method 3, `waste:common` |

### B. Failed attempts

The path to the working reset ran through several instructive dead-ends.

*1) Bulk transport:* The service-mode bulk-OUT timed out (errno 110) and bulk-IN returned only zero-length packets. The maintenance transport is not bulk at all (Section V); this consumed many cycles before the `usbprint.sys` decompile.

*2) Legacy clear opcode:* The earlier `85 00 00 00 03 01 03 07` payload was accepted (ACK 5) but never cleared 5B00 across a power cycle—the firmware accepts and ignores the unauthenticated write (Section III).

*3) Naive cipher framing:* An early encoder emitted "header ‖ 4-byte enciphered keyword" (10–12 bytes), the functor-3 read-handshake shape, not the genuine 23-byte write frame. It matched the genuine payload in only 2/20 bytes—below chance—confirming the role-swap correction of Section VII was necessary.

*4) Two-patch DRM bypass:* Patching only the two `QUERY_KEYS` gates left `RESET_GUID` to stall first; a keyed attempt froze at the licensing socket with zero USB writes until the third gate was added (Section VIII).

### C. Successful reset

With the corrected transport, protocol, and cipher, the native tool cleared 5B00 on the dedicated debug G6020. After the two writes and a clean power-button shutdown, the printer rebooted out of service mode and re-enumerated as `04a9:1865` (normal mode) instead of `04a9:12fe`. IPP then reported `printer-state = idle` with no `marker-waste-ink-receptacle-full` alert (Listing 7). The re-enumeration to `1865` is the at-a-glance signal that the printer left service mode healthy.

```
$ lsusb | grep 04a9
  Bus 001 Device 0NN: ID 04a9:1865 Canon # normal
      mode (was 12fe)
$ ipptool -tv ipp://printer/ get-attrs.test
  printer-state = idle (3)      # was stopped (5)
  printer-state-reasons = none  # 5B00 cleared
```

Listing 7. Post-reset: re-enumeration to normal mode and a clear IPP state.

## XII. Related Work

**Open Canon RE.** The Context IS "doomed encryption" work [4] and `leecher1337/pixma` [5] established firmware-unpack tooling but contain no maintenance/reset path; the SANE `pixma` backend [6] is a scanner backend on a different channel. No prior open tool reset a Canon G-series waste counter. **Open Epson lineage.** `reink` [7], `epson_print_conf` [8], and `epson-printer-snmp` [9] are the transferable prior art—the last derives its parameters by sniffing WICReset, the same method we apply. **Consumable DRM.** Cybowski's study of cartridge DRM [10] is the canonical capture–diff–reset writeup. **Printer/firmware security venues.** Cui *et al.* on firmware modification [11], Müller *et al.*'s systematization of printer attacks and PRET [12], and IoT firmware RE work [13], [14] situate the attack surface. **Tooling.** Frida [15], Ghidra [16], and the `IoctlHunter` hooking pattern [17] underpin the trifecta. **The commercial tool.** OctoInkjet's WICReset [2] is the cloud-gated product whose device-side reset this work proves is cloud-independent.

## XIII. Right-to-Repair Discussion

5B00 is a software counter, not a hardware fault: the absorber is mechanically serviceable (new pads or an external waste tank), and only the firmware counter blocks printing. The manufacturer offers no user reset [1], and the de facto remedy is a paid, network-validated, single-use key for a proprietary Windows tool [2]. Our decompilation shows the cloud contributes nothing to the device-side reset—it is licensing only—which reframes the gate as a business-model control rather than a technical necessity [18], [19]. The right-to-repair posture is explicit: the tool is clean-room and interoperability-focused; it operates only on a locked test unit until the protocol is verified; it redistributes no Canon firmware, no vendor binary, and no decompiler database; and physical servicing of the absorber is a documented precondition, since clearing the counter on a genuinely full pad risks ink overflow. We discuss the relevant DMCA §1201 software-repair exemption framing [20] as context for publishing the protocol, not as legal advice.

A further property strengthens this posture: the maintenance command set is *asymmetric*—it exposes a counter *clear* but no counter *set*. We establish this by triangulation, not assumption. First, an exhaustive static pass over the vendor Service Tool (`ServiceTool_v5103`) shows every absorber operation emits a fixed-literal payload `[00 03 flags 03 idx]` with no value field, and the binary contains *zero*

GetDlgItemInt/SetDlgItemInt references: no operation reads a numeric magnitude into a maintenance frame (the only value-bearing write, the serial number, scatters into a serial-only region and never reaches the absorber cell). Second, in the decrypted template the entire waste family is the selector 10 07 7C then 0D, a region byte, and 00; opcode 0x0D is hard-wired to clear-to-zero and the trailing value byte is invariantly 00—no set-to-value operand exists. Third, the recovered clear is idempotent and the counter register is read-only over this command set. The absorber counter can therefore be driven toward zero but never upward; recreating the 5B00 state would require firmware-level EEPROM access outside any host protocol, or physical wear. This asymmetry bounds the tool's blast radius—it can only *reduce* a maintenance counter, never inflate one to disable a unit—a desirable property for an interoperable repair tool.

## XIV. Comparison with Existing Approaches

Table IV compares the native reproduction to the commercial and manufacturer paths across practical dimensions.

The native reproduction is the only free, open, Linux-native, cloud-free, key-free option. It is most valuable to owners and repair technicians on their own fleets who want a reproducible, auditable reset rather than a metered black box.

## XV. Reproducibility and Artifact Availability

The work is reproducible from the open repository [3], which contains the native tool, the CANON-SR5 reference codec, the protocol specification, and the full RE notes that trace each claim from evidence to code. The repository is deliberately split into two halves: a *shippable* native reset tool (libusb resetter, safety gates, and a fleet playbook) and a *reproducibility-only* RE rig (the trace–decompile–correlate trifecta and its analysis utilities). Only the first is a product; the second exists so the result can be re-derived.

**The trifecta rig.** The three evidence lanes (Section IV)—host usbmon wire capture, Frida instrumentation inside the Win11 capture VM, and Ghidra decompilation—are enumerated reproducibly so an independent investigator can rebuild the workbench. The capture substrate is a throwaway Win11 guest under session-mode libvirt with real USB passthrough of the service-mode device; the three lanes are correlated by wall-clock timestamp and the deterministic 20-byte payload envelope.

**Provenance and the gated tool.** To respect provenance and licensing we redistribute no Canon firmware, no vendor binary, and no decompiler project database: the proprietary tools (Canon Service Tool, WICReset/Printer Potty, Canon firmware) are RE oracles only, never a production dependency, and every binary and capture they touch is excluded from version control. The RE notes instead cite the exact binary hashes (e.g. the printerpotty.exe SHA-256) and the static addresses of the cipher and the three licensing gates, so the decompile is independently reproducible without shipping the binary. The shippable tool itself is gated: it defaults to a dry run, and its destructive -execute path is locked to a registered test unit and an unpromoted protocol status until a verified capture is in hand (Section X).

**Validation artifacts.** The 23/23 cipher reproduction is checked by a test suite that runs two independent implementations against the genuine captured frame; the on-hardware reset was validated at a tagged commit. The deliberate negative result—that the 4th-byte keyword-padding rule cannot be generalized from a single sample (Section VII-C)—is documented so future work can target the controlled multi-sample capture campaign rather than overclaim from one frame. The paper itself builds reproducibly with Tectonic (Section XV-A).

### A. Build

The required class and style files (IEEEtran.cls, IEEEtran.bst, bytefield.sty, cleveref.sty, balance.sty) are vendored alongside this source so no system TeX install is needed:

```
$ cd docs/paper && tectonic canon-megatank-reset.
    tex
```

Listing 8. Reproducible build.

## XVI. Conclusion

We have presented an open, native-Linux, key-free, and cloud-free reproduction of the Canon MegaTank 5B00 waste-ink reset, validated on real hardware. The key findings are:

1) The service-mode maintenance transport is a USB *vendor* control transfer (0x41 OUT / 0xC1 IN, command byte in bRequest, whole frame as the data stage)—not a bulk transfer, which is why prior native attempts stalled.
2) The reset is an authenticated session: a plain set_session, a live per-session device keyword from get_keyword, and two enciphered 23-byte set_command writes; the get_command completion is empty by design and must not gate success.
3) The write cipher is a functor envelope with the buffer roles swapped (20-byte envelope as subject, 4-byte bound keyword as seed), validated byte-exact (23/23) against a genuine captured frame by two independent implementations.
4) The reset is cloud-independent: by decompilation, no cloud byte reaches the payload, keyword, or completion; the cloud is licensing only, proven by forcing the three licensing gates and observing identical, locally-derived frames on the wire.
5) The cleared counter commits on a clean power-button shutdown (printhead-park EEPROM flush), not on an abrupt unplug.

A software-counter-as-DRM gate that contributes nothing to the underlying device operation is, in the end, a licensing boundary and not a technical one—and reproducing the operation cleanly returns a serviceable printer to service.

TABLE IV
COMPARISON OF CANON MEGATANK 5B00 RESET APPROACHES

| Method | Platform | Cost / Key | Cloud | Open | Key Limitation |
|---|---|---|---|---|---|
| Native reproduction (this work) | Linux (libusb) | Free / none | No | Yes | Codec validated for G6000-series template |
| WICReset / Printer Potty [2] | Windows, Mac, Linux | Paid, single-use key | Yes (validate) | No | Per-printer, metered, online at reset |
| OctoInkjet G5 Potty kit [2] | (bundles WICReset) | Paid kit + key | Yes | No | Same reset engine as WICReset |
| Legacy service-mode clear | Native panel | Free | No | — | Disabled in firmware on G5/6/7000 |
| Manufacturer (Canon) | RMA | Warranty / replace | N/A | N/A | No user reset; replace device [1] |


## ACKNOWLEDGMENTS

The author thanks the `leecher1337/pixma` and Context IS firmware reverse-engineering lineage [4], [5], the open Epson waste-reset projects whose capture-and-correlate method this work mirrors [7]–[9], and the Frida and Ghidra communities [15], [16] whose tools made the trifecta possible.


## REFERENCES


[1] Canon U.S.A., Inc., "Support code 5b00 — Canon MegaTank: "service is required"," 2024, manufacturer position: no user reset; warranty/replace only — the unavailability this work addresses (ART143380). [Online]. Available: https://support.usa.canon.com/

[2] OctoInkjet, "WIC Reset / Printer Potty waste-ink counter reset," 2025, the cloud-gated commercial tool whose device-side reset this work proves is cloud-independent; genuine build PrinterPotty_WICReset-5.95. [Online]. Available: https://octoink.co.uk/getwicreset

[3] Tinyland, Inc., "canon-megatank-reset: open, native Linux Canon MegaTank waste-ink reset," 2026, this work. Hardware-validated native reset at commit d2f3c81; CANON-SR5 reference codec, safety-gate ops, protocol spec. [Online]. Available: https://github.com/Jesssullivan/canon-megatank-reset

[4] Context Information Security, "Hacking Canon PIXMA Printers — Doomed Encryption," 2016, foundational Canon firmware-update crypto RE; the "doomed encryption" lineage the pixma firmware-unpack tools build on. [Online]. Available: https://www.contextis.com/en/blog/hacking-canon-pixma-printers-doomed-encryption

[5] leecher1337, "pixma: Canon PIXMA firmware unpacking tools," 2020, firmware-unpack lineage (pixma_decrypt known-plaintext XOR, pixma_unpack Thumb decompressor, dec_sdata). No reset/service-mode code; no LICENSE file. The work this paper extends conceptually. [Online]. Available: https://github.com/leecher1337/pixma

[6] SANE Project, "SANE pixma backend (sane-pixma(5))," 2024, canon scanner command backend (bulk channel). Cited only as protocol-lineage prior art for Canon command framing; no waste-ink or maintenance surface. [Online]. Available: https://gitlab.com/sane-project/backends

[7] lion-simba (Elena), "reink: Epson ink-pad / waste-counter reset," 2012, iEEE-1284.4 (D4) EEPROM read/write to Epson maintenance counters; the canonical open Epson waste-reset reference. [Online]. Available: https://github.com/lion-simba/reink

[8] Ircama, "epson_print_conf: Epson printer configuration / waste reset via SNMP," 2024, ePSON-CTRL SNMP read/write-key frames, multi-byte LE counter with divider, temporary-vs-permanent commit semantics. [Online]. Available: https://github.com/Ircama/epson_print_conf

[9] Zedeldi, "epson-printer-snmp: Epson waste-ink reset over SNMP," 2023, parameters derived by sniffing WICReset — the same capture-and-correlate method used in this work. [Online]. Available: https://github.com/Zedeldi/epson-printer-snmp

[10] W. Cybowski, "Methods of implementation and operation of hardware DRM protection on the example of printer cartridges," *Technical Sciences*, vol. 25, pp. 117–137, 2022, peer-reviewed capture-diff-interpret-reset writeup for consumable DRM: modifies the on-chip data to reset protection and validates by observing the reported consumable level change. [Online]. Available: https://czasopisma.uwm.edu.pl/index.php/ts/article/view/7634

[11] A. Cui, M. Costello, and S. J. Stolfo, "When firmware modifications attack: A case study of embedded exploitation," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2013, canonical printer-firmware RE: HP LaserJet RFU format reversed and persistently modified — the upstream of printer-RE work.

[12] J. Müller, V. Mladenov, J. Somorovsky, and J. Schwenk, "SoK: Exploiting network printers," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2017, pp. 213–230, systematization of printer attacks and the PRET toolkit; formalizes NVRAM read/write at a named address — the abstract form of the service-mode write replayed here.

[13] O. Shwartz, Y. Mathov, M. Bohadana, Y. Elovici, and Y. Oren, "Reverse engineering IoT devices: Effective techniques and methods," *IEEE Internet of Things Journal*, vol. 5, no. 6, pp. 4965–4976, 2018, general bus-sniff / dump / diff method spine for chip-and-firmware RE, applied here to the service-mode counter hunt.

[14] T. Bakhshi, B. Ghita, and I. Kuzminykh, "A review of IoT firmware vulnerabilities and auditing techniques," *Sensors*, vol. 24, no. 2, p. 708, 2024, survey of firmware-RE / auditing tooling and the dump-diff workflow; taxonomy backdrop for the device-side counter hunt.

[15] Ole André V. Ravnås and contributors, "Frida: Dynamic instrumentation toolkit," 2024, used to hook DeviceIoControl and to patch the three cloud-licensing JZ gates (74 -> EB) for genuine-frame ground-truth capture. [Online]. Available: https://frida.re/

[16] National Security Agency, "Ghidra software reverse engineering framework," 2024, decompile of printerpotty.exe (WICReset) and usbprint.sys. [Online]. Available: https://ghidra-sre.org/

[17] Z4kSec, "IoctlHunter: hooking DeviceIoControl," 2023, reference pattern for the DeviceIoControl in/out-buffer hook. [Online]. Available: https://github.com/Z4kSec/IoctlHunter

[18] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Wiley, 2020, dRM / accessory-control chapter; economics of tying.

[19] ——, "Cryptography and competition policy: Issues with 'trusted computing'," in *Proceedings of the 22nd Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 2003, pp. 3–10, competition-policy analysis of consumable/accessory binding via cryptography; the business-model rationale behind a gated reset.

[20] B. T. Yeh, "Repair, modification, or resale of software-enabled consumer electronic devices: Copyright law issues," Congressional Research Service, CRS Report R44590, Aug. 2016, dMCA §1201 anti-circumvention and the first-sale / repair lens on resetting software maintenance locks (published 2016-08-11). [Online]. Available: https://crsreports.congress.gov/product/details?id=R44590